

ภาพรวมของบริการ (Service Overview) MissionProgress Service

GitHub: <https://github.com/Ratthatummanoon/CS366-MissionProgress-Service>

Video: <https://r2files.akawatmor.com/dp1/sfh/cs366/missionprogress/MissionProgress-Video.mp4>

1. Service Owner: รัฐธรรมนูญ โคสาแสง (6609612178)

2. Service Purpose

MissionProgress Service คือบริการสำหรับทีมกู้ภัย (Rescue Team) เพื่อใช้ในการรายงานความคืบหน้าของภารกิจที่ได้รับมอบหมาย อัปเดตสถานะของเหตุการณ์ (Incident Status) และบันทึกรายละเอียดการปฏิบัติงานหน้างาน (Action Logs) เพื่อให้ศูนย์สั่งการได้รับข้อมูลที่ถูกต้องและเป็นปัจจุบันที่สุด

3. Pain Point ที่แก้ไข

ปัจจุบันศูนย์สั่งการ (Command Center) ขาดการมองเห็นภาพรวมและการติดตามสถานะการทำงานของทีมกู้ภัยแบบเรียลไทม์ (Lack of real-time visibility) รวมถึงขาดข้อมูลประเมินความรุนแรง (Impact Assessment) ที่ถูกต้องแม่นยำจากหน้างานจริง ทำให้การตัดสินใจสั่งการหรือสนับสนุนทรัพยากรเกิดความล่าช้าและผิดพลาด

4. Target Users

- Rescue Team: ใช้สำหรับอัปเดตสถานะ แจ้งพิกัดเมื่อถึงหน้างาน และประเมินความรุนแรง
- Dispatcher: ใช้ดูข้อมูล Timeline การทำงานเพื่อติดตามผล (ผ่านการดึงข้อมูลไปแสดงผล)

5. Service Boundary

In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)

ความรับผิดชอบ	รายละเอียด
State Transition Management	รับผิดชอบการเปลี่ยนสถานะภารกิจตาม Workflow: DISPATCHED → EN_ROUTE → ON_SITE → NEED_BACKUP → RESOLVED พร้อม Validate ว่าการเปลี่ยนสถานะสมเหตุสมผล
Timeline / Action Log Recording	บันทึก Log การปฏิบัติงานทุกรายการ เช่น เวลาที่ถึงจุดเกิดเหตุ, การกระทำ (Evacuation start, First aid applied), ผู้กระทำ
Field Assessment Forwarding	รับข้อมูลประเมิน Impact Level / Priority จากทีมกู้ภัยหน้างาน บันทึกเป็น Action Log แล้ว Publish Event ไปยัง IncidentTracking (ผู้เป็นเจ้าของข้อมูล) เพื่ออัปเดตค่าจริง
Evidence Image Management	รับและจัดเก็บหลักฐานภาพถ่ายจากหน้างาน (Evidence Images) ผ่าน S3 Presigned URL (PUT, 5 นาที) พร้อมเชื่อมโยง Image Key กับ Timeline entry
Event Publishing	Publish Domain Events (MissionStatusChanged, MissionBackupRequested, ImpactLevelUpdated) ไปยัง Amazon EventBridge เพื่อแจ้ง Service อื่นๆ

Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)

สิ่งที่ไม่รับผิดชอบ	บริการที่รับผิดชอบ
การ "สั่งการ" หรือ "มอบหมายงาน" ให้ทีมกู้ภัย	Manage Dispatch Service
การค้นหาเส้นทาง	SafeRoute Service
การจัดการทรัพยากรโรงพยาบาล / การส่งตัวผู้ป่วย	HospitalResourceStatus Service
การเป็น Source of Truth ของ Impact Level / Priority / Location	IncidentTracking Service (MissionProgress แด่ Forward ข้อมูล)

6. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ

1. Status Validation (การตรวจสอบสถานะ)

- ตรวจสอบว่าการเปลี่ยนสถานะสมเหตุสมผลหรือไม่ตามตาราง State Transition ที่กำหนด เช่น ต้องเป็น ON_SITE ก่อนจึงจะ RESOLVED ได้ NEED_BACKUP จะ Trigger การ Publish Event เพื่อแจ้งเตือน

2. Field Assessment Acceptance (การรับข้อมูลจากหน้างาน)

- รับข้อมูล Impact Level / Priority ที่ทีมกู้ภัยปรับจากหน้างาน ได้ทันที โดยไม่ต้องรอการอนุมัติ
- บันทึก Timeline เป็น Action Log พร้อม Forward ผ่าน Event ให้ IncidentTracking อัปเดต

3. Idempotency Decision (การตัดสินใจเรื่อง Duplicate)

- ตรวจสอบ Idempotency Key ว่า Request นี้เคยถูกประมวลผลแล้วหรือยัง ถ้าเคย → Return cached response ทันที โดยไม่ประมวลผลซ้ำ

การตัดสินใจอิงจาก

แหล่งข้อมูล	รายละเอียด
Input จากทีมกู้ภัยหน้างาน	User Action
สถานะปัจจุบันของภารกิจ	Current State ใน DynamoDB
Idempotency Records	ใน DynamoDB

บริการตัดสินใจได้เองภายใต้ Business Rules ที่กำหนด โดยไม่ต้องรอการอนุมัติจากมนุษย์ในกรณีปกติ

7. Owned Data

ข้อมูลที่บริการนี้เป็นเจ้าของและดูแลโดยตรง (Source of Truth)

ข้อมูล	รายละเอียด
--------	------------

Mission Timeline / Action Logs	ข้อมูลประวัติการทำงานทั้งหมดที่เป็น Array of objects (เก็บ timestamp, action_type, description, performed_by, old_status, new_status, note, gps_location, image_key)
Operational Context Updates	ข้อมูลบริบทงานล่าสุดที่ทีมกู้ภัยส่งมา เช่น last_updated_at, latest_impact_level, priority_level (Rescue Unit ID)

8. Linked Data (Reference Only)

ข้อมูลที่ใช้บริการนี้ต้องอ้างอิงจากบริการอื่น แต่ไม่ได้เป็นเจ้าของหลัก

ข้อมูล	แหล่งอ้างอิง	วัตถุประสงค์
Incident Master Data	IncidentTracking Service (หรือ Core Incident Schema)	อ้างอิง incident_id, incident_type, incident_description เพื่อแสดงผลให้ทีมกู้ภัยดู
Rescue Team Info	ระบบจัดการทีม	อ้างอิง ID ของทีมกู้ภัย เพื่อระบุว่าใครเป็นคนส่ง Log

9. Non-Functional Requirements

NFR	รายละเอียด	สอดคล้องกับ Architecture อย่างไร
High Availability (99.9%)	ระบบต้องพร้อมใช้งาน 24/7 หยุดชะงักไม่ได้	Lambda + DynamoDB + S3 + EventBridge ทั้งหมดเป็น AWS Managed Services มี built-in HA หลาย AZ
Low Latency (<500ms)	Update Status และ Read Timeline ต้องตอบสนอง <500ms	Go Lambda cold start ~80-150ms, DynamoDB single-digit ms, API GW ~10-20ms → รวม ~100-200ms ปกติ
Concurrent Handling	รองรับหลายร้อยทีมพร้อมกันในช่วงวิกฤต	Lambda auto-scales (concurrent executions), DynamoDB On-Demand auto-scales (no capacity planning)
Data Integrity	Timeline ต้องเรียงลำดับเวลาถูกต้อง (Sequential Consistency)	DynamoDB Sort Key ใช้ ISO 8601 timestamp → Query ด้วย ScanIndexForward=true ได้เรียงตามเวลา
Resilience & Idempotency	รองรับ Network หลุดชั่วคราว, Client Retry ได้โดยไม่เกิด Duplicate	Client-side: เก็บ Pending Actions ใน localStorage, Retry ด้วย Idempotency Key เดิม — Server-side: Lambda ตรวจสอบ Idempotency Key ใน DynamoDB ด้วย attribute_not_exists(PK) Condition → ป้องกัน Duplicate แบบ Atomic — Event-level: EventBridge

		Publish ล้มเหลว → Outbox Pattern → outbox-processor retry
Data Durability	ข้อมูลต้องไม่สูญหาย	DynamoDB 99.99% durability, S3 99.99% durability, EventBridge + Outbox Pattern ป้องกัน Event loss

Synchronous Function Contract

Base URL: https://{URL}/mission-progress/v1

Response Header Convention (ใช้กับทุก API): ทุก Response จาก MissionProgress Service จะมี Header ดังนี้

Header	คำอธิบาย
Content-Type	application/json
X-Trace-Id	Unique Trace ID (UUID v4) สำหรับติดตาม request ให้อ้างอิงเมื่อแจ้งปัญหา
Access-Control-Allow-Origin	*
Access-Control-Allow-Methods	GET,POST,OPTIONS
Access-Control-Allow-Headers	x-api-key,X-Rescue-Team-ID,Content-Type

ยกเว้น 403 Forbidden ที่ return จาก API Gateway Authorizer โดยตรง — จะไม่มี X-Trace-Id เนื่องจากไม่ผ่าน application layer

Error Response Convention (ใช้กับทุก API): ทุก Error Response (ยกเว้น 403) จะมีโครงสร้างเดียวกัน

```
{
  "error": "ERROR_CODE",
  "code": "ERROR_CODE",
  "message": "Human-readable error description",
  "traceld": "550e8400-e29b-41d4-a716-446655440000"
}
```

Field	Type	คำอธิบาย
error	String	Error code สำหรับ programmatic handling
code	String	Error code (เหมือน error)
message	String	ข้อความอธิบายปัญหาที่อ่านเข้าใจง่าย
traceld	String	Trace ID ตัวเดียวกับ X-Trace-Id ใน header ใช้แจ้ง debug

API Contract #1: Report Progress & Update Status

รายการ	ค่า
Name	reportMissionProgress
Method	POST
Path	/missions/{request_id}/progress
Type	Synchronous
Lambda	report-progress (Go)

คำอธิบาย

ใช้สำหรับทีมกู้ภัยเพื่อ "บันทึกการปฏิบัติงาน" (Create new Timeline entry) และ "อัปเดตสถานะ" (Update mission status) ของภารกิจในครั้งเดียว เช่น แจ้งว่าถึงจุดเกิดเหตุแล้ว, ขอกำลังเสริม, หรือแจ้งปิดงาน พร้อมทั้ง Publish Events ไป EventBridge เพื่อแจ้ง Service อื่นๆ

Request

Path Parameters

Parameter	Type	Required	คำอธิบาย
request_id	String		รหัส request จาก RescueRequest Service (เช่น REQ-001)

Headers

Header	Required	คำอธิบาย
Content-Type		application/json
x-api-key		API Key สำหรับ Authentication
X-Rescue-Team-ID		รหัสทีมกู้ภัยที่ส่งข้อมูล (เช่น TEAM-ALPHA)

Body

{ "new_status": "ON_SITE", "note": "ถึงจุดเกิดเหตุแล้ว กำลังเริ่มปฐมพยาบาลเบื้องต้น", "new_impact_level": "HIGH" }
--

Field	Type	Required	คำอธิบาย
-------	------	----------	----------

new_status	String (Enum)	✓	สถานะใหม่: EN_ROUTE, ON_SITE, NEED_BACKUP, RESOLVED
note	String	✗	รายละเอียดการปฏิบัติงาน / หมายเหตุ
new_impact_level	Integer	✗	ระดับความรุนแรงใหม่จากการประเมินหน้างาน (ตัวเลข) → publish ImpactLevelUpdated event
current_location	String	✗	พิกัดปัจจุบันของทีม (GPS string) บันทึกลง Timeline entry
image_key	String	✗	S3 Key ของรูปภาพหลักฐาน (ได้จาก presigned-url API) → เชื่อม Evidence กับ Timeline entry

Response

Success 200 OK

Response Headers

```
Content-Type: application/json
X-Trace-Id: 550e8400-e29b-41d4-a716-446655440000
```

Response Body

```
{
  "message": "Progress reported successfully",
  "mission_id": "MSN-001",
  "request_id": "REQ-001",
  "old_status": "EN_ROUTE",
  "new_status": "ON_SITE",
  "updated_at": "2025-06-14T09:32:15Z"
}
```

Error 400 Bad Request State Transition ผิดกฎ

```
{
  "error": "INVALID_STATE_TRANSITION",
  "code": "INVALID_STATE_TRANSITION",
  "message": "Cannot transition from EN_ROUTE to RESOLVED",
  "traceId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

Error 400 Bad Request ไม่ส่ง new_status

```
{
  "error": "MISSING_PARAMETER",
  "code": "MISSING_PARAMETER",
  "message": "new_status is required",
  "traceId": "b2c3d4e5-f6a7-8901-bcde-f12345678901"
}
```

Error 400 Bad Request สถานะไม่รู้จัก

```
{
  "error": "INVALID_STATUS",
  "code": "INVALID_STATUS",
  "message": "Invalid status value: UNKNOWN_STATUS",
  "traceId": "c3d4e5f6-a7b8-9012-cdef-123456789012"
}
```

Error 400 Bad Request JSON body ไม่ถูกต้อง

```
{
  "error": "INVALID_REQUEST_BODY",
  "code": "INVALID_REQUEST_BODY",
  "message": "Invalid request body",
  "traceId": "d4e5f6a7-b8c9-0123-defa-234567890123"
}
```

Error 404 Not Found ไม่พบ request_id

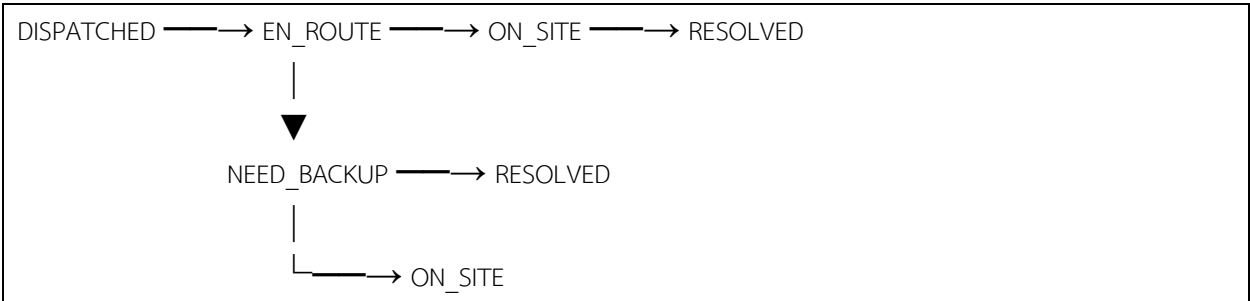
```
{
  "error": "MISSION_NOT_FOUND",
  "code": "MISSION_NOT_FOUND",
  "message": "No mission found for request: REQ-99999",
  "traceId": "e5f6a7b8-c9d0-1234-efab-345678901234"
}
```

Error 403 Forbidden — Auth ไม่ผ่าน

Response นี้มาจาก API Gateway โดยตรง ไม่มี X-Trace-Id header และไม่มี traceId ใน body


```
{
  "message": "Forbidden"
}
```

State Machine (Validation Rules)



จาก	ไปได้	ไปไม่ได้
DISPATCHED	EN_ROUTE	❌ ห้ามขึ้นตอนไม่ได้
EN_ROUTE	ON_SITE	❌ RESOLVED, NEED_BACKUP
ON_SITE	NEED_BACKUP, RESOLVED	❌ EN_ROUTE, DISPATCHED
NEED_BACKUP	ON_SITE, RESOLVED	❌ EN_ROUTE, DISPATCHED
RESOLVED	❌ (Final State)	❌ ทุกสถานะ

Dependency / Reliability

ประเภท	รายละเอียด
Internal Data (Write)	อัปเดต MissionAssignment table (current_status, last_updated_at) + Insert entry ใน MissionTimeline table
EventBridge (Async)	Publish สูงสุด 3 Events: MissionStatusChanged (ทุกครั้ง), MissionBackupRequested (ถ้า NEED_BACKUP), ImpactLevelUpdated (ถ้ามี new_impact_level)
Outbox Fallback	หาก EventBridge Publish ล้มเหลว → บันทึกลง EventOutbox table → POST request ไม่ fail (ข้อมูลสถานะ safe ใน DynamoDB แล้ว)
Validation	ตรวจสอบ State Transition ตาม Business Rules ก่อนบันทึก reject ถ้าไม่ถูกกฎ

API Contract #2: View Incident Mission Details

รายการ	ค่า
Name	getMissionDetails
Method	GET
Path	/missions/{request_id}
Type	Synchronous
Lambda	get-mission (Go)

คำอธิบาย

ใช้ดึงข้อมูลรายละเอียดของภารกิจ รวมถึง "ประวัติการทำงานทั้งหมด (Timeline)" และ "ข้อมูล Request จาก RescueRequest Service" (Degraded Mode เมื่อเรียกไม่สำเร็จ) เพื่อให้ทีมกู้ภัยอุ่นหลังหรือ Dispatcher ตรวจสอบความคืบหน้า

Request

Path Parameters

Parameter	Type	Required	คำอธิบาย
request_id	String		รหัส request จาก RescueRequest Service (เช่น REQ-001)

Headers

Header	Required	คำอธิบาย
x-api-key		API Key สำหรับ Authentication
X-Rescue-Team-ID		รหัสทีมกู้ภัย

Body

ไม่มี

Response

Success 200 OK - Degraded Mode

Response Headers

Content-Type: application/json
X-Trace-Id: f6a7b8c9-d0e1-2345-fab0-456789012345

Response Body

```
{
  "request_id": "REQ-001",
  "mission_id": "MSN-001",
  "rescue_team_id": "TEAM-ALPHA",
  "current_status": "ON_SITE",
  "latest_impact_level": 3,
  "started_at": "2024-12-01T08:00:00Z",
  "last_updated_at": "2025-06-14T09:32:15Z",
  "timeline": [
    {
      "mission_id": "MSN-001",
      "timestamp": "2024-12-01T08:00:00Z",
      "log_id": "LOG-001",
      "action_type": "STATUS_CHANGE",
      "description": "Mission dispatched to TEAM-ALPHA",
      "performed_by": "SYSTEM"
    },
    {
      "mission_id": "MSN-001",
      "timestamp": "2025-06-14T09:32:15Z",
      "log_id": "LOG-002",
      "action_type": "STATUS_CHANGE",
      "description": "ถึงจุดเกิดเหตุแล้ว",
      "old_status": "EN_ROUTE",
      "new_status": "ON_SITE",
      "performed_by": "TEAM-ALPHA"
    }
  ],
  "data_source": "partial"
}
```

Success 200 OK — Full Mode (สถานะปัจจุบัน)

```
{
```

```
{
  "request_id": "REQ-001",
  "mission_id": "MSN-001",
  "rescue_team_id": "TEAM-ALPHA",
  "current_status": "ON_SITE",
  "latest_impact_level": 3,
  "started_at": "2024-12-01T08:00:00Z",
  "last_updated_at": "2025-06-14T09:32:15Z",
  "description": "น้ำท่วมหนักบริเวณถนนพหลโยธิน",
  "location": "13.7563,100.5018",
  "incident_type": "FLOOD",
  "timeline": ["..."],
  "data_source": "full"
}
```

Field Availability by Mode

Field	มี Degraded?	มี Full?	Source
request_id, mission_id, rescue_team_id	✓	✓	MissionAssignment table
current_status, latest_impact_level	✓	✓	MissionAssignment table
started_at, last_updated_at	✓	✓	MissionAssignment table
timeline	✓	✓	MissionTimeline table
description, location, incident_type	✗	✓	RescueRequest Service
data_source	"partial"	"full"	—

Error 404 Not Found

```
{
  "error": "MISSION_NOT_FOUND",
  "code": "MISSION_NOT_FOUND",
  "message": "No mission found for request: REQ-99999",
  "traceld": "11112222-3333-4444-5555-666677778888"
}
```

Error 403 Forbidden

Response นี้มาจาก API Gateway โดยตรง — ไม่มี X-Trace-Id header และไม่มี traceld ใน body

```
{  
  "message": "Forbidden"  
}
```

Dependency / Reliability

ประเภท	รายละเอียด
Internal Data (Read)	อ่าน MissionAssignment table (state) + MissionTimeline table (timeline เรียงตาม timestamp)
External Dependency	HTTP GET ไปยัง 3 services แบบ parallel (sync.WaitGroup, timeout 800ms + retry 2x ต่อ service): RescueRequest (GET /v1/rescue-requests/{requestId}), ManageDispatch (GET /v1/dispatches?teamId={teamId}), RescueTeam (GET /v1/teams/{teamId})
Degraded Mode	Service ใด service หนึ่งล้ม/timeout → data_source: "partial", omit empty fields หายไปจาก response
Full Mode Condition	เรียก RescueRequest + ManageDispatch + RescueTeam สำเร็จทั้ง 3 → data_source: "full"

API Contract #3: Request Presigned URL for Evidence Upload

รายการ	ค่า
Name	requestEvidenceUploadURL
Method	POST
Path	/missions/{request_id}/presigned-url
Type	Synchronous
Lambda	presigned-url (Go)

คำอธิบาย

ใช้สำหรับทีมกู้ภัยเพื่อ "ขอ Presigned URL" จากระบบ ก่อนอัปโหลดรูปภาพหลักฐานจากหน้างาน (Evidence Image) ตรงไปยัง Amazon S3 โดยไม่ต้องส่งไฟล์ผ่าน Lambda (ลดภาระ Server + หลีกเลี่ยง Lambda payload limit 6MB)

Flow การใช้งาน

1. Frontend เรียก POST /missions/{id}/presigned-url → ส่ง file_name + content_type
2. Lambda สร้าง Presigned URL (PUT) อายุ 5 นาที → ส่งกลับ upload_url + image_key
3. Frontend ใช้ upload_url อัปโหลดไฟล์ตรงไป S3 (HTTP PUT)
4. Frontend ส่ง image_key ไปพร้อม POST /missions/{id}/progress (เพื่อเชื่อม Evidence กับ Timeline entry)

Request

Path Parameters

Parameter	Type	Required	คำอธิบาย
request_id	String	✓	รหัส request จาก RescueRequest Service (เช่น REQ-001)

Headers

Header	Required	คำอธิบาย
Content-Type	✓	application/json
x-api-key	✓	API Key สำหรับ Authentication
X-Rescue-Team-ID	✓	รหัสทีมกู้ภัยที่ส่งข้อมูล (เช่น TEAM-ALPHA)

Body

```
{
  "file_name": "flood-evidence-001.jpg",
  "content_type": "image/jpeg"
}
```

Field	Type	Required	คำอธิบาย
file_name	String	✓	ชื่อไฟล์ที่ต้องการอัปโหลด (เช่น photo-001.jpg)
content_type	String	✓	MIME type ของไฟล์ (เช่น image/jpeg, image/png)

Response

Success 200 OK

Response Headers

```
Content-Type: application/json
X-Trace-Id: aabb1122-ccdd-3344-eeff-556677889900
```

Response Body

```
{
  "upload_url": "https://s3.amazonaws.com/mission-evidence-bucket/evidence/MSN-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg?X-Amz-Algorithm=AWS4-HMAC-SHA256&...",
  "image_key": "evidence/MSN-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg",
  "expires_in": 300,
  "message": "Presigned URL generated successfully"
}
```

Field	Type	คำอธิบาย
upload_url	String	Presigned URL สำหรับ HTTP PUT อัปโหลดไฟล์ตรงไป S3
image_key	String	S3 Key ของไฟล์ — ใช้แนบไปกับ POST /progress เพื่อเชื่อมกับ Timeline
expires_in	Integer	อายุของ URL เป็นวินาที (300 = 5 นาที)
message	String	ข้อความยืนยัน

S3 Key Format

```
evidence/{mission_id}/{rescue_team_id}/{unix_timestamp}-{file_name}
```

ตัวอย่าง: evidence/MSN-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg

ใช้ {unix_timestamp} เพื่อป้องกันชื่อไฟล์ซ้ำ

Error 400 Bad Request — ไม่ส่ง field ที่จำเป็น

```
{
  "error": "INVALID_REQUEST_BODY",
  "code": "INVALID_REQUEST_BODY",
  "message": "file_name and content_type are required",
  "traceld": "11aa22bb-33cc-44dd-55ee-66ff77008811"
}
```

Error 400 Bad Request — content_type ไม่รองรับ

```
{
  "error": "INVALID_CONTENT_TYPE",
  "code": "INVALID_CONTENT_TYPE",
  "message": "Supported content types: image/jpeg, image/png, image/webp",
  "traceld": "22bb33cc-44dd-55ee-66ff-778800991122"
}
```

Error 404 Not Found — ไม่พบ request_id

```
{
  "error": "MISSION_NOT_FOUND",
  "code": "MISSION_NOT_FOUND",
  "message": "No mission found for request: REQ-99999",
  "traceld": "33cc44dd-55ee-66ff-7788-009911223344"
}
```

Error 403 Forbidden

Response นี้มาจาก API Gateway โดยตรง — ไม่มี X-Trace-Id header และไม่มี traceld ใน body

```
{
```



```
"message": "Forbidden"
}
```

Frontend Upload Flow (หลังได้ Presigned URL)

```
# Frontend ใช้ upload_url ที่ได้มา อัปโหลดไฟล์ตรงไป S3
curl -X PUT \
  -H "Content-Type: image/jpeg" \
  --data-binary @flood-evidence-001.jpg \
  "https://s3.amazonaws.com/mission-evidence-bucket/evidence/REQ-001/...?X-Amz-Algorithm=..."

# HTTP 200 = อัปโหลดสำเร็จ

# จากนั้นส่ง image_key ไปพร้อม POST /progress
curl -X POST \
  -H "x-api-key: $API_KEY" \
  -H "X-Rescue-Team-ID: TEAM-ALPHA" \
  -H "Content-Type: application/json" \
  -d '{
    "new_status": "ON_SITE",
    "note": "ถึงจุดเกิดเหตุ น้ำสูง 1.2m",
    "image_key": "evidence/MSN-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg"
  }' \
  "$API_URL/missions/REQ-001/progress"
```

Dependency / Reliability

ประเภท	รายละเอียด
Internal Data (Read)	ตรวจสอบว่า request_id มี Mission อยู่ใน MissionAssignment table
AWS S3	สร้าง Presigned URL (PUT) ด้วย AWS SDK — ไม่ได้ upload ไฟล์จริงใน step นี้
Validation	ตรวจสอบ file_name ไม่ว่าง, content_type เป็นรูปภาพที่รองรับ (jpeg/png/webp)
Security	Presigned URL อายุ 5 นาที → หหมดอายุแล้วใช้ไม่ได้ ต้องขอใหม่
Upload Failure	ถ้า Frontend อัปโหลดไป S3 ไม่สำเร็จ → อนุญาตให้ "ข้าม" (Skip) → ส่งเฉพาะ Text Status ก่อนได้

API Contract #4: List Team Missions

รายการ	ค่า
Name	listTeamMissions
Method	GET
Path	/missions
Type	Synchronous
Lambda	list-missions (Go)

คำอธิบาย

ใช้ดึง รายการภารกิจทั้งหมด ของทีมกู้ภัยที่ระบุ เพื่อให้ทีมกู้ภัยเห็นภาพรวมว่าตัวเองมีกี่ภารกิจ แต่ละภารกิจอยู่สถานะอะไร → กดเข้าไปดู Timeline ละเอียดด้วย GET /missions/{request_id} ต่อได้

Use Cases

- ทีมกู้ภัยเปิดแอปมา → เห็นรายการภารกิจทั้งหมดของตัวเอง
- Dispatcher ดูว่าทีมหนึ่งๆ รับผิดชอบภารกิจอะไรบ้าง
- กรอง Active missions (ยังไม่ RESOLVED) เพื่อโฟกัสงานที่ต้องทำ

Request

team_id ถูกดึงจาก X-Rescue-Team-ID header (ผ่าน Lambda Authorizer context) — ไม่ใช่ query parameter

Query Parameters

Parameter	Type	Required	คำอธิบาย
status	String	✗	กรองเฉพาะสถานะ (เช่น ON_SITE, NEED_BACKUP) ถ้าไม่ส่ง = ดึงทุกสถานะ

Headers

Header	Required	คำอธิบาย
x-api-key	✓	API Key สำหรับ Authentication
X-Rescue-Team-ID	✓	รหัสทีมกู้ภัย

Body

ไม่มี

ตัวอย่าง Request

```
# ดึงภารกิจทั้งหมดของ TEAM-ALPHA
curl -s -H "x-api-key: $API_KEY" \
  -H "X-Rescue-Team-ID: TEAM-ALPHA" \
  "$API_URL/missions?team_id=TEAM-ALPHA" | jq .

# ดึงเฉพาะภารกิจที่สถานะ ON_SITE
curl -s -H "x-api-key: $API_KEY" \
  -H "X-Rescue-Team-ID: TEAM-ALPHA" \
  "$API_URL/missions?team_id=TEAM-ALPHA&status=ON_SITE" | jq .
```

Response

Success 200 OK

Response Headers

```
Content-Type: application/json
X-Trace-Id: 99887766-5544-3322-1100-aabbccddeeff
```

Response Body

```
{
  "team_id": "TEAM-ALPHA",
  "total_missions": 3,
  "missions": [
    {
      "mission_id": "MSN-001",
      "request_id": "REQ-001",
      "current_status": "ON_SITE",
      "latest_impact_level": 3,
      "started_at": "2024-12-01T08:00:00Z",
      "last_updated_at": "2025-06-14T09:32:15Z"
    },
  ],
}
```

```
{
  "mission_id": "MSN-003",
  "request_id": "REQ-003",
  "current_status": "NEED_BACKUP",
  "latest_impact_level": 5,
  "started_at": "2025-06-14T07:00:00Z",
  "last_updated_at": "2025-06-14T10:15:00Z"
},
{
  "mission_id": "MSN-005",
  "request_id": "REQ-005",
  "current_status": "RESOLVED",
  "latest_impact_level": 2,
  "started_at": "2025-06-13T14:00:00Z",
  "last_updated_at": "2025-06-13T18:30:00Z"
}
]
```

Field	Type	คำอธิบาย
team_id	String	รหัสทีมกู้ภัยที่ค้นหา
total_missions	Integer	จำนวนภารกิจทั้งหมดที่พบ
missions	Array	รายการภารกิจ (สรุป ไม่รวม Timeline)
missions[].mission_id	String	รหัสภารกิจ
missions[].request_id	String	รหัสเหตุการณ์
missions[].current_status	String	สถานะปัจจุบัน
missions[].latest_impact_level	Integer	ระดับความรุนแรงล่าสุด
missions[].started_at	String (ISO 8601)	เวลาเริ่มภารกิจ
missions[].last_updated_at	String (ISO 8601)	เวลาอัปเดตล่าสุด

Success 200 OK — ไม่พบภารกิจ

```
{
  "team_id": "TEAM-UNKNOWN",
```

```
"total_missions": 0,  
"missions": []  
}
```

Success 200 OK — กรองด้วย status

```
{  
  "team_id": "TEAM-ALPHA",  
  "total_missions": 1,  
  "missions": [  
    {  
      "mission_id": "MSN-001",  
      "request_id": "REQ-001",  
      "current_status": "ON_SITE",  
      "latest_impact_level": 3,  
      "started_at": "2024-12-01T08:00:00Z",  
      "last_updated_at": "2025-06-14T09:32:15Z"  
    }  
  ]  
}
```

Error 400 Bad Request — ไม่ส่ง X-Rescue-Team-ID

```
{  
  "error": "MISSING_PARAMETER",  
  "code": "MISSING_PARAMETER",  
  "message": "X-Rescue-Team-ID is required",  
  "traceId": "44556677-8899-aabb-ccdd-eeff00112233"  
}
```

Error 400 Bad Request — status ไม่ถูกต้อง

```
{  
  "error": "INVALID_STATUS",  
  "code": "INVALID_STATUS",  
}
```

```

"message": "Invalid status filter: UNKNOWN_STATUS. Valid values: DISPATCHED, EN_ROUTE, ON_SITE,
NEED_BACKUP, RESOLVED",
"traceId": "55667788-99aa-bbcc-ddee-ff0011223344"
}

```

Error 403 Forbidden

Response นี้มาจาก API Gateway โดยตรง — ไม่มี X-Trace-Id header และไม่มี traceId ใน body

```

{
  "message": "Forbidden"
}

```

DynamoDB Query Strategy

ใช้ GSI team-index ของ MissionAssignment table

```

GSI: team-index
  Partition Key: rescue_team_id = "TEAM-ALPHA"

→ ได้ภารกิจทั้งหมดของทีม

ถ้ามี status filter:
→ Filter Expression: current_status = :status
→ DynamoDB filter หลัง query (ไม่ใช่ Key Condition)

```

Dependency / Reliability

ประเภท	รายละเอียด
Internal Data (Read)	Query MissionAssignment table ผ่าน GSI team-index (Partition Key = rescue_team_id)
No External Dependency	ไม่เรียก Service อื่น — อ่านจาก DynamoDB ของตัวเองเท่านั้น
Validation	ตรวจสอบ team_id ไม่ว่าง, status (ถ้ามี) เป็นค่าที่ถูกต้อง
Performance	GSI query = single-digit ms / Filter Expression ทำที่ DynamoDB → ไม่กระทบ Lambda
Empty Result	ไม่พบภารกิจ → return 200 OK พร้อม missions: [] (ไม่ใช่ 404)

Asynchronous Function Contract

Message Contract #1: MissionStatusChanged

Field	Value
Message Name	MissionStatusChangedEvent
Interaction Style	Asynchronous (Publish/Subscribe)
Producer	MissionProgress Service (report-progress Lambda — Go)
Consumers	IncidentTracking, Dispatch Management
Channel	EventBridge (mission-progress-events)

คำอธิบาย

Event ถูก publish เมื่อมีการเปลี่ยนสถานะภารกิจสำเร็จ (ผ่าน validation)

ใช้สำหรับ:

- IncidentTracking → อัปเดตสถานะ Incident
- Dispatch → ปลดล็อกทีม (เฉพาะ RESOLVED)

Trigger: POST /incidents/{id}/progress

Message Format

```
{
  "source": "MissionProgressService",
  "detail-type": "MissionStatusChanged",
  "detail": {
    "schema_version": "1.0",
    "mission_id": "MSN-001",
    "requestId": "REQ-8812-9901",
    "incident_id": "INC-001",
    "rescue_team_id": "TEAM-ALPHA",
    "old_status": "EN_ROUTE",
    "new_status": "ON_SITE",
    "changed_at": "2025-06-14T09:32:15Z",
    "changed_by": "TEAM-ALPHA"
  }
}
```

Field	Type	Required	Description
source	String	✓	MissionProgressService
detail-type	String	✓	MissionStatusChanged
detail.schema_version	String	✓	Schema version (current: "1.0")
detail.mission_id	String	✓	รหัสภารกิจ
detail.requestId	String	✓	รหัส request จาก RescueRequest
detail.incident_id	String	✓	รหัสเหตุการณ์
detail.rescue_team_id	String	✓	ทีมกู้ภัย
detail.old_status	String	✓	สถานะเดิม
detail.new_status	String	✓	สถานะใหม่
detail.changed_at	ISO 8601 String	✓	เวลาที่เปลี่ยนสถานะ
detail.changed_by	String	✓	ผู้กระทำ (Rescue Team ID)

Validation Rules

- new_status ต้องถูกต้องตาม State Machine
- changed_at ต้องเป็น ISO 8601
- rescue_team_id ห้ามว่าง

Consumer Routing

Consumer	Rule Filter	สถานะปัจจุบัน
IncidentTracking	detail-type = MissionStatusChanged	EventBridge Direct (Lambda)
ManageDispatch	(sync PATCH on RESOLVED)	Sync PATCH (fire-and-forget)

Failure Handling

- Publish ล้มเหลว → Outbox Pattern
- POST ไม่ fail
- EventBridge retry อัตโนมัติ (24 ชม.)

Message Contract #2: MissionBackupRequested

Field	Value
Message Name	MissionBackupRequestedEvent
Producer	MissionProgress Service
Consumers	Prioritization Service
Channel	EventBridge

คำอธิบาย

Event เมื่อสถานะเป็น NEED_BACKUP

ใช้สำหรับ:

Prioritization → คำนวณ Priority ใหม่

Message Format

{
"source": "MissionProgressService",
"detail-type": "MissionBackupRequested",
"detail": {
"schema_version": "1.0",
"mission_id": "MSN-001",
"incident_id": "INC-001",
"rescue_team_id": "TEAM-ALPHA",
"requested_at": "2025-06-14T10:15:00Z",
"requested_by": "TEAM-ALPHA",
"location": "13.7563,100.5018"
}
}

Field	Required	Description
detail.schema_version		Schema version (current: "1.0")
detail.mission_id		รหัสภารกิจ
detail.incident_id		รหัสเหตุการณ์
detail.rescue_team_id		ทีมกู้ภัย

detail.requested_at	✓	ISO 8601 เวลาที่ขอ backup
detail.requested_by	✓	ผู้ขอ backup (Rescue Team ID)
detail.location	✗	GPS string หน้ํางาน (optional)

Consumer Routing

Consumer	Rule Filter	สถานะปัจจุบัน
Prioritization	MissionBackupRequested	SQS Route Active

Failure Handling

- Outbox Pattern
- Non-blocking

Message Contract #3: ImpactLevelUpdated

Field	Value
Message Name	ImpactLevelUpdatedEvent
Consumers	IncidentTracking + Prioritization

คำอธิบาย

Event เมื่อมีการปรับ Impact Level จากหน้างาน

Message Format

{
"source": "MissionProgressService",
"detail-type": "ImpactLevelUpdated",
"detail": {
"schema_version": "1.0",
"mission_id": "MSN-001",
"incident_id": "INC-001",
"rescue_team_id": "TEAM-ALPHA",
"old_level": 2,
"new_level": 4,
"updated_at": "2025-06-14T09:35:00Z",
"updated_by": "TEAM-ALPHA"
}
}

Field	Type	Required	Description
detail.old_level	Integer	✓	ระดับความรุนแรงเดิม (capture ก่อน update)
detail.new_level	Integer	✓	ระดับความรุนแรงใหม่จากหน้างาน
detail.updated_at	ISO 8601	✓	เวลา
detail.updated_by	String	✓	ผู้ปรับ (Rescue Team ID)

หมายเหตุ: Event นี้จะถูก publish เฉพาะเมื่อ new_level != old_level — ใส่ค่าเดิมซ้ำจะไม่ถูกส่ง

Consumer Routing

Consumer	Rule Filter	สถานะปัจจุบัน
IncidentTracking	ImpactLevelUpdated	EventBridge Direct (Lambda)
Prioritization	ImpactLevelUpdated	SQS Route Active

Failure Handling

- Outbox Pattern
- EventBridge retry
- Non-blocking - ทีมกู้ภัยยังทำงานได้แม้ Event ส่งไม่ถึง

Message Contract #4: DispatchOrderCreated (Inbound)

Field	Value
Message Name	DispatchOrderCreated
Interaction Style	Asynchronous (Subscribe)
Producer	Manage Dispatch Service
Consumer	MissionProgress Service (mission-assigned-handler Lambda — Go)
Channel	SNS (rescue.mission.dispatch.v1)

คำอธิบาย

MissionProgress ฟัง Event นี้จาก Manage Dispatch Service เมื่อ Dispatcher มอบหมายงานให้ทีมกู้ภัย Lambda mission-assigned-handler จะสร้าง MissionAssignment record และ Timeline entry แรกอัตโนมัติ

ผลลัพธ์

- สร้าง MissionAssignment (status = DISPATCHED)
- สร้าง MissionTimeline entry แรก (action_type = MISSION_ASSIGNED)
- ดึง incident_id จาก RescueRequest Service (degraded: empty string ถ้าล้มเหลว)
- Idempotency check ด้วย dispatch_id — ถ้า mission มีอยู่แล้ว → skip

Expected Payload (จาก Manage Dispatch Service via SNS)

Manage Dispatch ส่ง message มาผ่าน SNS Topic rescue.mission.dispatch.v1 ด้วย envelope ดังนี้

```
{
  "header": {
    "messageType": "DispatchOrderCreated",
    "traceId": "trace-uuid"
  },
  "body": {
    "dispatchId": "DSP-001",
    "requestId": "REQ-001",
    "teamId": "TEAM-ALPHA",
    "priorityLevel": "HIGH",
    "status": "DISPATCHED",
    "dispatchedAt": "2025-06-14T08:45:00Z",
    "timestamp": "2025-06-14T08:45:00Z"
  }
}
```

```

}
}

```

Field	Type	Required	Description
header.messageType	String	✓	ต้องเป็น "DispatchOrderCreated" (filter ใน handler)
header.traceId	String	✗	สำหรับ distributed tracing
body.dispatchId	String	✓	รหัส Dispatch Order (idempotency key)
body.requestId	String	✓	รหัส request จาก RescueRequest
body.teamId	String	✓	รหัสทีมกู้ภัย
body.priorityLevel	String	✗	"CRITICAL" / "HIGH" / "NORMAL" / "LOW"
body.status	String	✗	สถานะ Dispatch Order
body.dispatchedAt	String	✗	เวลาที่มอบหมาย (ISO 8601)

Failure Handling

กรณี	การจัดการ
Duplicate event	Idempotency check → skip (ไม่ error)
RescueRequest ล้ม	Degraded Mode → incident_id = "" (ยังสร้าง mission)
Missing required fields	Return error → SNS retry
Unknown messageType	Log + skip (ไม่ error)

Service Data

1) Mission Timeline Data (Owned by this service)

ข้อมูลประวัติการทำงานของทีมกู้ภัย (Log) แต่ละภารกิจ ใช้สำหรับสร้าง Timeline

DynamoDB Table: MissionTimeline | PK: mission_id | SK: timestamp | GSI: log-id-index (hash: log_id)

Field Name	Type	Required	Description	Example
mission_id	String	Yes	รหัสภารกิจ (Partition Key)	MISS-a1b2c3d4
timestamp	DateTime	Yes	เวลาที่เกิดเหตุการณ์จริง (Sort Key)	2024-10-15T12:30:00Z
log_id	String	Yes	UUID ของรายการ Log (GSI key)	550e8400-e29b-41d4-a716-446655440000
action_type	String	Yes	ประเภทการกระทำ: STATUS_CHANGE, MISSION_ASSIGNED, UPLOAD	STATUS_CHANGE
description	String	Yes	รายละเอียดของสิ่งที่ทำ (auto-generated: "Status changed: X → Y")	Status changed: EN_ROUTE → ON_SITE
performed_by	String	Yes	ID ของทีมกู้ภัยหรือ SYSTEM	TEAM-01
old_status	String	No	สถานะเดิม (เฉพาะ STATUS_CHANGE)	EN_ROUTE
new_status	String	No	สถานะใหม่ (เฉพาะ STATUS_CHANGE)	ON_SITE
note	String	No	หมายเหตุเพิ่มเติมจากทีมกู้ภัย	ถึงจุดเกิดเหตุแล้ว น้ำสูง 1.2m
gps_location	String	No	พิกัดจากทีม (เนื้อหาจาก current_location field)	13.7563,100.5018
image_key	String	No	S3 Key ของรูปภาพหลักฐาน (จาก presigned-url API)	evidence/MISS-abc/TEAM-01/1718352735-x.jpg

Notes:

- Timeline entries จะถูกเรียงตาม timestamp (Sort Key)
- action_type = MISSION_ASSIGNED เมื่อ mission-assigned-handler สร้าง Mission ใหม่
- image_key เชื่อม Evidence รูปภาพเข้ากับ Timeline entry สำหรับดึง view URL ภายหลัง
- gps_location จะถูกเก็บใน DynamoDB ด้วย attribute name gps_location

2) Mission Assignment State (Owned by this service)

ข้อมูลสถานะปัจจุบันของภารกิจที่ทีมกู้ภัยแต่ละทีมรับผิดชอบ

DynamoDB Table: MissionAssignment | PK: mission_id | GSI: request-index (request_id), team-index (rescue_team_id), dispatch-index (dispatch_id)

Field Name	Type	Required	Description	Example
mission_id	String	Yes	รหัสภารกิจ (Primary Key, รูปแบบ: MISS-{uuid8})	MISS-a1b2c3d4
dispatch_id	String	Yes	รหัส Dispatch Order จาก Manage Dispatch Service (GSI key)	DSP-001
request_id	String	Yes	รหัส request จาก RescueRequest Service (lookup key, GSI)	REQ-8812-9901
incident_id	String	Yes	รหัสเหตุการณ์ (ดึงจาก RescueRequest Service ตอนเกิด)	INC-12345
rescue_team_id	String	Yes	รหัสทีมกู้ภัยที่รับผิดชอบภารกิจนี้ (GSI key)	TEAM-01
priority_level	Integer	No	ลำดับความสำคัญจาก Manage Dispatch Service	2
current_status	String	Yes	สถานะปัจจุบันของภารกิจ	ON_SITE
latest_impact_level	Integer	No	ระดับความรุนแรงล่าสุดที่ประเมินโดยทีมนี้	3
started_at	DateTime	Yes	เวลาที่เริ่มรับภารกิจ (จาก dispatchedAt)	2024-10-15T12:00:00Z
last_updated_at	DateTime	Yes	เวลาที่อัปเดตข้อมูลล่าสุด	2024-10-15T12:35:00Z

Notes:

- current_status ค่าที่ถูกต้อง: DISPATCHED, EN_ROUTE, ON_SITE, NEED_BACKUP, RESOLVED
- latest_impact_level = 0 ถ้ายังไม่มี การประเมิน (Integer, ไม่ใช่ null)
- request_id คือ primary lookup key — ใช้ค้นหา mission จาก path parameter {request_id}
- dispatch_id ใช้เป็น idempotency key ใน mission-assigned-handler
- 1 incident_id มีได้หลาย request_id → incident_id ไม่ unique สำหรับ lookup
- description, location, requestType ดึงจาก RescueRequest Service แบบ on-demand (degraded mode ถ้า service ไม่พร้อม)

3) Event Outbox (Owned by this service)

ใช้สำหรับ Outbox Pattern — บันทึก EventBridge events ที่ยังไม่ได้ส่ง หรือส่งไม่สำเร็จ ให้ outbox-processor Lambda retry

DynamoDB Table: EventOutbox | PK: outbox_id | SK: created_at | GSI: status-index (status) | TTL: enabled

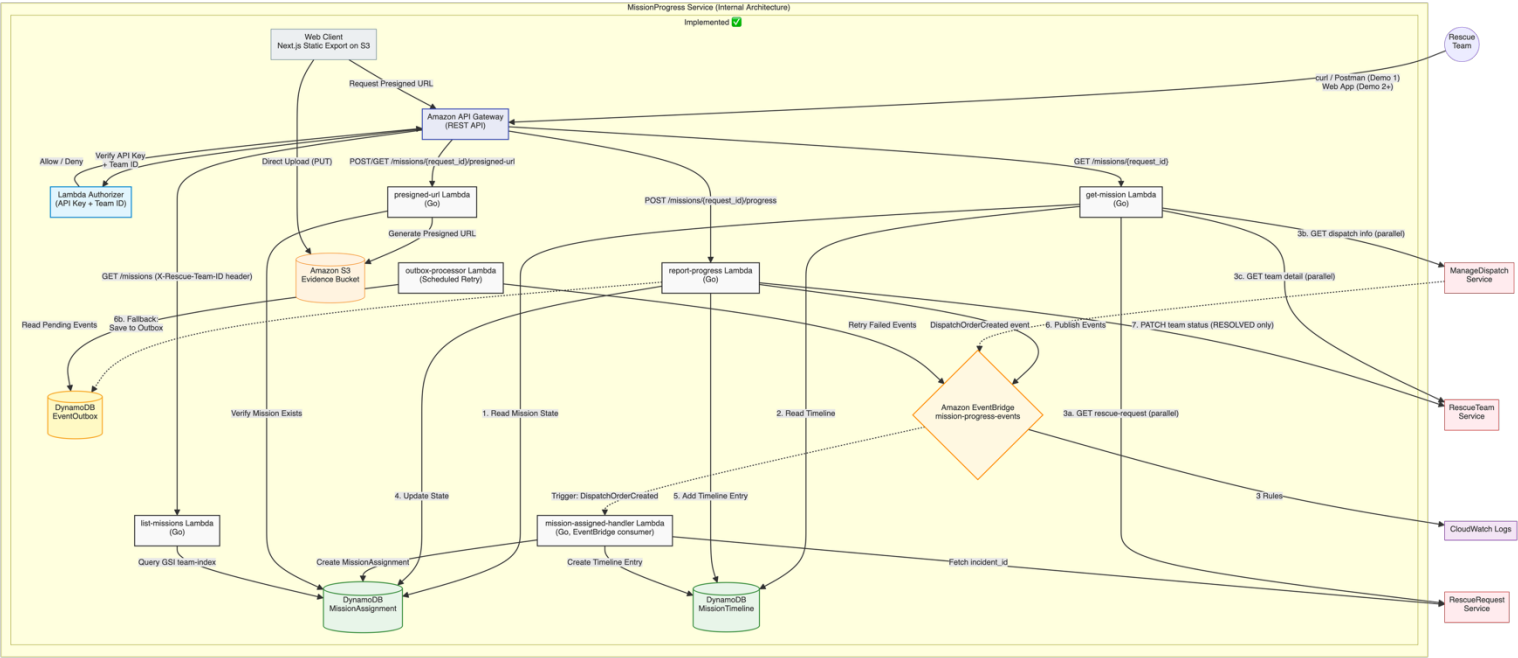
Field Name	Type	Required	Description	Example
outbox_id	String	Yes	UUID ของ outbox record (Partition Key)	550e8400-e29b-41d4-a716-...
created_at	String	Yes	ISO 8601 เวลาที่สร้าง record (Sort Key)	2025-06-14T09:32:15Z
event_type	String	Yes	ชื่อ detail-type ของ event	MissionStatusChanged
event_payload	String	Yes	JSON string ของ event detail	{"mission_id": "MISS-a1b2..."}
status	String	Yes	สถานะการส่ง: PENDING, SENT, FAILED	PENDING
retry_count	Integer	Yes	จำนวนครั้งที่ retry (max 5)	0
ttl	Integer	No	Unix timestamp สำหรับ DynamoDB TTL auto-delete	1718438535

Notes:

- outbox-processor Lambda ดึง records ที่ status = PENDING/FAILED และ retry_count < 5
- เมื่อส่ง EventBridge สำเร็จ → อัปเดต status = SENT
- เมื่อ retry ครบ 5 ครั้ง → status = FAILED (dead letter)
- TTL ช่วยลบ SENT records อัตโนมัติ

Service Architecture

Architecture Diagram



Components

1. Amazon API Gateway (REST API)

Overview

- Single Entry Point สำหรับทุก HTTP Request
- Routing ตาม Path + Method ไปยัง Lambda ที่เกี่ยวข้อง

Routes

Method	Path	Auth	Target Lambda
GET	/missions/{request_id}	CUSTOM	get-mission
POST	/missions/{request_id}/progress	CUSTOM	report-progress
POST	/missions/{request_id}/presigned-url	CUSTOM	presigned-url
GET	/missions/{request_id}/presigned-url?image_key=	CUSTOM	presigned-url
GET	/missions (header: X-Rescue-Team-ID)	CUSTOM	list-missions
GET	/health	NONE	health-check

2. Lambda Authorizer (AuthN + AuthZ)

หน้าที่ ตรวจสอบ Header

- x-api-key (Authentication)
- X-Rescue-Team-ID (Authorization)

Behavior

กรณี	ผลลัพธ์
ถูกต้อง	ผ่านไปยัง Lambda
ไม่ถูกต้อง / ไม่มี	403 Forbidden

ไม่เรียก Core Lambda → ประหยัด cost

3. get-mission Lambda (Go)

หน้าที่

- ดึง Mission State + Timeline
- เรียก 3 external services แบบ Parallel (sync.WaitGroup)

External Calls (Parallel)

Service	Endpoint	ข้อมูลที่ได้
RescueRequest	GET /v1/rescue-requests/{requestId}	description, location, requestType, peopleCount
ManageDispatch	GET /v1/dispatches?teamId={teamId}	dispatch_status, priority_level
RescueTeam	GET /v1/teams/{teamId}	team_name, team_type, capabilities, equipment

Behavior

กรณี	ผลลัพธ์
ทั้ง 3 services สำเร็จ	data_source: full
Service ใดล้มเหลว (timeout)	data_source: partial

4. report-progress Lambda (Go)

หน้าที่

- Validate State Machine
- Update DynamoDB

- Publish Events → EventBridge

Flow

- Validate Transition

- Update MissionAssignment

- Insert MissionTimeline

- Publish Events (สูงสุด 3 events)

- Fallback → Outbox Pattern

5. Amazon DynamoDB

Tables

Table	PK	SK	GSI
MissionAssignment	mission_id	—	request-index (request_id), team-index (rescue_team_id), dispatch-index (dispatch_id)
MissionTimeline	mission_id	timestamp	log-id-index (log_id)
EventOutbox	outbox_id	created_at	status-index (status), TTL enabled

Notes

- ใช้ On-Demand (PAY_PER_REQUEST)

- dispatch-index ใช้ใน mission-assigned-handler สำหรับ idempotency check

6. Amazon EventBridge

Overview

- Custom Bus: mission-progress-events

- ใช้สำหรับ Event-driven communication

Events

Event	Trigger	สถานะปัจจุบัน
MissionStatusChanged	ทุกครั้งที่สถานะเปลี่ยน	Active (IncidentTracking: EventBridge Direct; Dispatch: SQS)
MissionBackupRequested	NEED_BACKUP	SQS Route Active (Prioritization)
ImpactLevelUpdated	มี new_impact_level	Active (IncidentTracking: EventBridge Direct; Prioritization: SQS)

7. External Services

RescueRequest Service

Lambda	Endpoint	Timeout	Retry
get-mission	GET /v1/rescue-requests/{requestId}	800ms	2
mission-assigned-handler	GET /v1/rescue-requests/{requestId}	800ms	2

ManageDispatch Service

Lambda	Endpoint	Timeout	Retry
get-mission	GET /v1/dispatches?teamId={teamId}	800ms	2

RescueTeam Service

Lambda	Endpoint	Timeout	Retry
get-mission	GET /v1/teams/{teamId}	800ms	2
report-progress	PATCH /v1/teams/{teamId}/status	800ms	2 (fire-and-forget on RESOLVED)

8. Web Client (Next.js)

- Static Export (deploy บน S3)
- รองรับ Mobile
- ไม่ต้อง install app

9. presigned-url Lambda

หน้าที่

- POST: Generate S3 Presigned URL สำหรับ upload (PUT, 5 นาที)
- GET: Generate S3 Presigned URL สำหรับ view (GET, 5 นาที)

Validation (POST)

- file_name ต้องมี
- content_type ต้องเป็น image/jpeg, image/png, หรือ image/webp

S3 Key Format

- evidence/{mission_id}/{rescue_team_id}/{unix_timestamp}-{file_name}

Output

- POST: { upload_url, image_key, expires_in }
- GET: { view_url, image_key, expires_in }

10. list-missions Lambda

หน้าที่

- Query ภารกิจของทีมผ่าน team-index GSI
- Team ID มาจาก X-Rescue-Team-ID header (ไม่ใช่ query param)

Behavior

กรณี	Response
พบข้อมูล	missions[]
ไม่พบ	200 OK + []
ไม่มี header	400 MISSING_PARAMETER

11. S3 Evidence Bucket

- เก็บรูปภาพหลักฐาน
- Upload ผ่าน Presigned URL (PUT)
- View ผ่าน Presigned URL (GET)

12. outbox-processor Lambda

หน้าที่

- Retry EventBridge events ที่ล้มเหลว (Scheduled)

Flow

- Scan status = PENDING/FAILED และ retry_count < 5
- Retry → EventBridge
- Update status → SENT / FAILED (ถ้าครบ 5 ครั้ง)

13. mission-assigned-handler Lambda

หน้าที่

- EventBridge consumer สำหรับ DispatchOrderCreated จาก Manage Dispatch Service

- สร้าง MissionAssignment และ Timeline entry แรก

Flow

- รับ DispatchOrderCreated event
- Idempotency check ด้วย dispatch_id (GSI dispatch-index)
- ดึง incident_id จาก RescueRequest Service (degraded ถ้าล้มเหลว)
- สร้าง MissionAssignment (status = DISPATCHED, mission_id = MISS-{uuid8})
- สร้าง Timeline entry (action_type = MISSION_ASSIGNED)

Explanation

1. Authentication Flow

ทุก request ต้องมี

- x-api-key
- X-Rescue-Team-ID

ผลลัพธ์	Behavior
	ผ่าน
	403

2. GET Flow — Mission + Timeline

GET /missions/{request_id}

ดึง:

- MissionAssignment (DynamoDB)
- MissionTimeline (DynamoDB)
- RescueRequest data (parallel HTTP)
- ManageDispatch data (parallel HTTP)
- RescueTeam data (parallel HTTP)

Key Behavior

- ทั้ง 3 services สำเร็จ → full
- Service ใดล้มเหลว → partial (Degraded Mode)

3. POST Flow — Update + Event

POST /progress

Steps

- Validate State
- Update DB
- Insert Timeline
- Publish Events
- Fallback → Outbox

State Transition

จาก	ไปได้
DISPATCHED	EN_ROUTE
EN_ROUTE	ON_SITE
ON_SITE	NEED_BACKUP, RESOLVED
NEED_BACKUP	ON_SITE, RESOLVED
RESOLVED	❌ Final State

Event Mapping

Event	Trigger	Target
MissionStatusChanged	ทุกครั้ง	SQS → IncidentTracking + Dispatch
MissionBackupRequested	NEED_BACKUP	SQS → Prioritization
ImpactLevelUpdated	มี impact	SQS → IncidentTracking + Prioritization

4. Reliability & Failure Handling

Level 1 Degraded Mode

IncidentTracking ล้มเหลว → ใช้ข้อมูล local

Level 2 Outbox Pattern



SENT / FAILED

Level 3 Authorizer ล้ม

API Gateway → 500

Level 4 Client Resilience (Demo 2+)

- เก็บ request ใน localStorage
- retry เมื่อ online

5. Evidence Upload Flow

Step

- ขอ Presigned URL
- Upload → S3
- ส่ง image_key กลับมา

ข้อดี

- รองรับไฟล์ใหญ่ (≤ 5GB)
- ลดภาระ Lambda

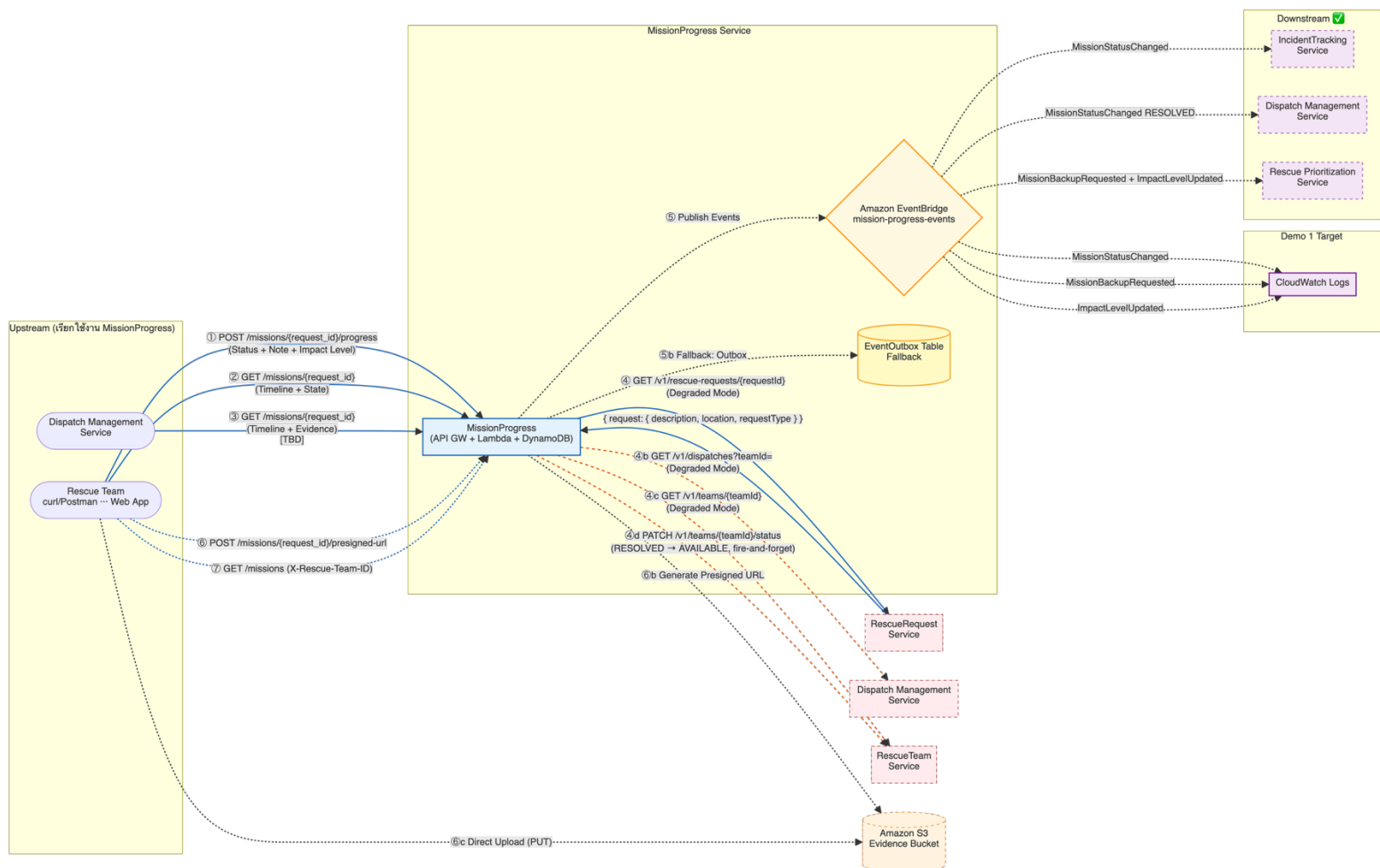
6. List Missions Flow

- Query ผ่าน GSI team-index
- รองรับ filter status

Behavior

กรณี	Response
มีข้อมูล	missions[]
ไม่มี	[]

Service Interaction Diagram



คำอธิบาย Flow

สัญลักษณ์	ความหมาย
เส้นทึบ (==)	Synchronous — รอผลตอบกลับทันที (HTTP Request/Response)
เส้นประ (- - -)	Asynchronous — ส่งแล้วไม่รอผล (Event ผ่าน EventBridge)

Upstream Services (บริการที่เรียกใช้งาน MissionProgress)

① POST /missions/{request_id}/progress — Rescue Team

รายละเอียด	ค่า
Purpose	รายงานสถานะล่าสุดจากหนงาน
Method	POST
Auth	x-api-key + X-Rescue-Team-ID

Lambda	report-progress (Go)
Client	curl/Postman, Web App

Request Body

```
{
  "new_status": "ON_SITE",
  "note": "ถึงจุดเกิดเหตุแล้ว น้ำสูง 1.2m",
  "new_impact_level": 3, // optional Integer
  "current_location": "13.7563,100.5018", // optional String
  "image_key": "evidence/MISS-abc/TEAM-ALPHA/1718352735-x.jpg" // optional
}
```

Response 200

```
{
  "message": "Progress reported successfully",
  "mission_id": "MISS-a1b2c3d4",
  "request_id": "REQ-001",
  "incident_id": "INC-001",
  "old_status": "EN_ROUTE",
  "new_status": "ON_SITE",
  "updated_at": "2025-..."
}
```

② GET /missions/{request_id} — Rescue Team

รายละเอียด	ค่า
Purpose	ดึง Timeline + สถานะปัจจุบัน
Method	GET
Auth	x-api-key + X-Rescue-Team-ID
Lambda	get-mission (Go)

Response 200

```
{
  "request_id": "REQ-001",
```

```

"incident_id": "INC-001",
"mission_id": "MSN-001",
"dispatch_id": "DSP-001",
"rescue_team_id": "TEAM-ALPHA",
"team_name": "Alpha Rescue Unit",
"team_type": "FLOOD",
"capabilities": ["swift_water_rescue", "first_aid"],
"equipment": ["boat", "lifejackets"],
"team_location": { "lat": 13.7563, "lng": 100.5018 },
"priority_level": 2,
"dispatch_status": "ACTIVE",
"current_status": "ON_SITE",
"latest_impact_level": 3,
"started_at": "2024-12-01T08:00:00Z",
"last_updated_at": "2025-06-14T09:32:15Z",
"description": "น้ำท่วมหนักบริเวณถนนพหลโยธิน",
"location": "13.7563,100.5018",
"incident_type": "FLOOD",
"timeline": [ ... ],
"data_source": "full"
}

```

data_source: "full" — เรียก RescueRequest + ManageDispatch + RescueTeam ครบ

data_source: "partial" — service ใดservice หนึ่งล้มเหลว (Degraded Mode) — omit empty fields จะหายไปจาก

response

③ GET /missions/{request_id} — Dispatch Management Service

รายละเอียด	ค่า
Purpose	Dispatcher ดู Timeline ละเอียด + รูปภาพหลักฐาน
Method	GET (ใช้ endpoint เดียวกับ ②)

ทำไม Dispatch อาจต้องเรียก GET API (ไม่ใช่แค่ฟัง Event)

ข้อมูลที่ Dispatcher ต้องการ	ได้จาก Event?	ได้จาก GET API?
------------------------------	---------------	-----------------

สถานะปัจจุบัน	✓	✓
Timeline ทั้งหมดย้อนหลัง	✗ (Event ส่งแค่ entry ล่าสุด)	✓
รูปภาพหลักฐาน (Demo 2+)	✗	✓

⑥ POST /missions/{request_id}/presigned-url — Rescue Team

รายละเอียด	ค่า
Purpose	ขอ Presigned URL สำหรับอัปโหลดรูปหลักฐาน
Method	POST
Auth	x-api-key + X-Rescue-Team-ID
Lambda	presigned-url (Go)

Request Body

```
{
  "file_name": "flood-evidence-001.jpg",
  "content_type": "image/jpeg"
}
```

Response 200

```
{
  "upload_url": "https://s3.amazonaws.com/...",
  "image_key": "evidence/REQ-001/TEAM-ALPHA/1718352735-flood-evidence-001.jpg",
  "expires_in": 300,
  "message": "Presigned URL generated successfully"
}
```

Full Upload Flow

- ⑥ a. Frontend → POST /missions/{request_id}/presigned-url → Lambda
- ⑥ b. Lambda → S3 (Generate PUT Presigned URL) → Return upload_url + image_key
- ⑥ c. Frontend → S3 (Direct PUT Upload ด้วย Presigned URL)
- ⑥ d. Frontend → POST /missions/{request_id}/progress (แนบ image_key) → เชื่อม Evidence กับ Timeline entry

GET /missions/{request_id}/presigned-url?image_key= — Rescue Team

รายละเอียด	ค่า
Purpose	ขอ Presigned URL สำหรับดูรูปหลักฐาน
Method	GET
Auth	x-api-key + X-Rescue-Team-ID
Lambda	presigned-url (Go)
Query	image_key=evidence/MISS-abc/TEAM-01/...

Response 200

```
{
  "view_url": "https://s3.amazonaws.com/...",
  "image_key": "evidence/MISS-abc/TEAM-ALPHA/1718352735-flood-evidence-001.jpg",
  "expires_in": 300,
  "message": "Presigned URL generated successfully"
}
```

⑦ GET /missions — Rescue Team

รายละเอียด	ค่า
Purpose	ดึงรายการภารกิจทั้งหมดของทีม
Method	GET
Auth	x-api-key + X-Rescue-Team-ID
Lambda	list-missions (Go)

Query Parameters

Parameter	Required	Description
status		กรอง เช่น ON_SITE, NEED_BACKUP

Team ID มาจาก X-Rescue-Team-ID header — ไม่ใช่ query parameter

Response 200

```
{
  "team_id": "TEAM-ALPHA",
```

```

"total_missions": 3,
"missions": [
  {
    "mission_id": "MSN-001",
    "request_id": "REQ-001",
    "current_status": "ON_SITE",
    "latest_impact_level": 3,
    "started_at": "2024-12-01T08:00:00Z",
    "last_updated_at": "2025-06-14T09:32:15Z"
  },
  ...
]
}

```

⑨ DispatchOrderCreated — จาก Dispatch Management Service (Inbound Async via SNS)

รายละเอียด	ค่า
Source	Dispatch Management Service
Trigger	เมื่อ Dispatcher มอบหมายภารกิจให้ทีมกู้ภัย
Channel	SNS (rescue.mission.dispatch.v1)
Lambda	mission-assigned-handler (Go)

Expect Payload

```

{
  "header": {
    "messageType": "DispatchOrderCreated",
    "traceId": "trace-uuid"
  },
  "body": {
    "dispatchId": "DSP-001",
    "requestId": "REQ-001",
    "teamId": "TEAM-ALPHA",
    "priorityLevel": "HIGH",
    "status": "DISPATCHED",

```

```
"dispatchedAt": "2025-06-14T08:45:00Z",
"timestamp": "2025-06-14T08:45:00Z"
}
}
```

mission-assigned-handler จะทำอะไร

- 1. Idempotency check ด้วย dispatch_id (GSI dispatch-index) — ถ้า mission มีอยู่แล้ว → skip
- 2. ดึง incident_id จาก RescueRequest Service (degraded ถ้าล้มเหลว: incident_id = "")
- 3. สร้าง MissionAssignment (status = DISPATCHED, mission_id = MISS-{uuid8})
- 4. สร้าง Timeline entry (action_type = MISSION_ASSIGNED)

Downstream Services (บริการปลายทางที่ MissionProgress เรียกหรือส่งข้อมูลไป)

MissionProgress สื่อสารกับ Downstream ผ่าน 2 ช่องทาง:

Synchronous (HTTP) — เรียกดึงข้อมูลแบบรอผลตอบกลับ

Asynchronous (EventBridge Events) — ส่ง Event แล้วไม่รอผล

1. RescueRequest Service

Owner: Phattharaphum Kingchai บทบาท: เป็น Source of Truth ของข้อมูล Request (description, location, requestType)

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด
④	Sync GET /v1/rescue-requests/{requestId} (Bearer)	MissionProgress → RescueRequest	ดึง description, location, requestType (ถ้าล้มเหลว → Degraded Mode, data_source: "partial")
⑤a	Async MissionStatusChanged	MissionProgress → RescueRequest	แจ้งอัปเดตสถานะ Request ที่เชื่อมโยง

Expected API Response (Sync GET)

```
{
  "request": {
    "requestId": "REQ-001",
```



```

    "description": "น้ำท่วมหนักบริเวณถนนพหลโยธิน",
    "location": {
      "latitude": 13.7563,
      "longitude": 100.5018
    },
    "requestType": "FLOOD"
  }
}

```

Failure Handling

กรณี	การจัดการ
Sync GET ล้มเหลว (timeout 800ms + retry 2x)	Degraded Mode — ส่งเฉพาะข้อมูลที่มี → ทีมกู้ภัยยังทำงานได้ปกติ
Async Event Publish ล้มเหลว	Outbox Pattern → บันทึกลง EventOutbox → outbox-processor retry ส่งใหม่

2. IncidentTracking Service

Owner: Krittamet Damthongkam บทบาท: รับ Async Events จาก MissionProgress (ไม่มี Sync call แล้ว)

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด
⑤a	Async MissionStatusChanged	MissionProgress → IncidentTracking	อัปเดตสถานะรวมของ Incident (เช่น "In Progress")
⑤b	Async ImpactLevelUpdated	MissionProgress → IncidentTracking	ส่ง Impact Level ล่าสุด → IncidentTracking อัปเดต MD

Failure Handling

กรณี	การจัดการ
Async Event Publish ล้มเหลว	Outbox Pattern → EventOutbox table → outbox-processor retry
EventBridge → Incident Target ล้มเหลว	EventBridge built-in retry (24 ชั่วโมง)

3. Dispatch Management Service

Owner: Noppakron Songkroh บทบาท: จัดการการมอบหมายงานและสถานะทีมกู้ภัย (BUSY/AVAILABLE)

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด
④b	Sync GET /v1/dispatches?teamId={teamId}	MissionProgress → Dispatch	ดึง dispatch status, priority level (ล้มเหลว → Degraded Mode, omit จาก response)
⑤b	Sync PATCH /v1/dispatches/{id}/status	MissionProgress → Dispatch	แจ้งว่าภารกิจ RESOLVED → ปิด Dispatch Order (fire-and-forget, ไม่ block response)

Expected API Response (④b Sync GET)

```
{
  "teamId": "TEAM-ALPHA",
  "items": [
    {
      "dispatchId": "DSP-001",
      "requestId": "REQ-001",
      "status": "ACTIVE",
      "priorityLevel": "HIGH",
      "dispatchedAt": "2025-06-14T08:45:00Z"
    }
  ]
}
```

PATCH Request Body (⑤b Sync PATCH on RESOLVED)

```
{
  "status": "RESOLVED",
  "teamId": "TEAM-ALPHA",
  "note": "Mission completed by MissionProgress Service"
}
```

หมายเหตุ: Dispatch ยังเป็น Upstream ด้วย — ส่ง DispatchOrderCreated event เข้ามาผ่าน SNS (⑨) + อาจเรียก GET API (③)

Failure Handling

กรณี	การจัดการ
Sync GET ล้มเหลว	Degraded Mode → omit dispatch_status, priority_level จาก response
EventBridge Publish ล้มเหลว	Outbox Pattern → EventOutbox table → outbox-processor retry
EventBridge → Dispatch Target ล้มเหลว	EventBridge built-in retry (24 ชั่วโมง)

4. Rescue Prioritization Service

Owner: Nattasak Chonmanat บทบาท: จัดลำดับความสำคัญและความเร่งด่วนของแต่ละเคส (Priority Score)

#	ช่องทาง	ทิศทาง	รายละเอียด
⑤c	Async MissionBackupRequested	MissionProgress → Prioritization	ทีมกู้ภัยต้องการกำลังเสริม (NEED_BACKUP) → คำนวณ Priority Score ใหม่
⑤d	Async ImpactLevelUpdated	MissionProgress → Prioritization	ส่ง Impact Level ล่าสุด → คำนวณลำดับ ความสำคัญใหม่

Failure Handling

กรณี	การจัดการ
EventBridge Publish ล้มเหลว	Outbox Pattern (safety net สำหรับ retry)
Event ส่งไม่ถึง Prioritization	Non-blocking — ระบบกู้ภัยยังทำงานต่อได้ (ไม่ใช่ Critical Path)

5. Amazon S3 — Evidence Bucket

บทบาท: เก็บรูปภาพหลักฐานจากหน้างาน

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด
⑥b	Sync Generate Presigned URL	presigned-url Lambda → S3	สร้าง Presigned URL (PUT) อายุ 5 นาที
⑥c	Frontend → S3 Direct	Rescue Team → S3	อัปโหลดรูปตรงไป S3 ด้วย Presigned URL (ไม่ผ่าน MissionProgress)

Failure Handling

กรณี	การจัดการ
------	-----------

Presigned URL Generation ล้มเหลว	Lambda return 500 → ให้ผู้ใช้ retry
Upload ไป S3 ล้มเหลว	อนุญาตให้ "ข้าม" → ส่งเฉพาะ Text Status ก่อน

6. RescueTeam Service

Owner: กมลพันธุ์ กันธายอด บทบาท: เป็น Source of Truth ของข้อมูลทีมกู้ภัย (ชื่อ, ประเภท, ความสามารถ, อุปกรณ์, ตำแหน่ง)

การสื่อสาร

#	ช่องทาง	ทิศทาง	รายละเอียด
④c	Sync GET /v1/teams/{teamId} (Bearer)	MissionProgress → RescueTeam	ดึง team_name, team_type, capabilities, equipment, location (ล้มเหลว → Degraded Mode)
④d	Sync PATCH /v1/teams/{teamId}/status (Bearer)	MissionProgress → RescueTeam	เมื่อ new_status = RESOLVED → ส่ง {"status": "AVAILABLE"} เพื่อปล่อยทีม (best-effort, fire-and-forget goroutine)

Expected API Response (④c Sync GET)

<pre>{ "teamId": "TEAM-ALPHA", "teamName": "Alpha Rescue Unit", "teamType": "FLOOD", "capabilities": ["swift_water_rescue", "first_aid"], "equipment": ["boat", "lifejackets"], "location": { "lat": 13.7563, "lng": 100.5018 } }</pre>

Failure Handling

กรณี	การจัดการ
Sync GET ล้มเหลว	Degraded Mode → omit team_name, team_type, capabilities, equipment, team_location จาก response

Sync PATCH ลมเหลว	Best-effort — log แล้วผ่าน ไม่ block response กลับ caller (fire-and-forget goroutine)
-------------------	---

EventBridge Events — Routing Detail

⑤ Lambda → EventBridge (mission-progress-events)

report-progress Lambda publish events เมื่อ POST สำเร็จ

Event	Trigger	Payload สำคัญ
MissionStatusChanged	ทุกครั้งที่สถานะเปลี่ยน	mission_id, request_id, rescue_team_id, old_status, new_status, changed_at, changed_by
MissionBackupRequested	new_status = NEED_BACKUP	mission_id, incident_id, rescue_team_id, requested_at, requested_by, location (optional)
ImpactLevelUpdated	มี new_impact_level ใน body	mission_id, incident_id, rescue_team_id, old_level, new_level, updated_at, updated_by

EventBridge Event Payload ตัวอย่าง

MissionStatusChanged

<pre>{ "source": "MissionProgressService", "detail-type": "MissionStatusChanged", "detail": { "schema_version": "1.0", "mission_id": "MISS-a1b2c3d4", "requestId": "REQ-001", "incident_id": "INC-001", "rescue_team_id": "TEAM-ALPHA", "old_status": "EN_ROUTE", "new_status": "ON_SITE", "changed_at": "2025-06-14T09:32:15Z", "changed_by": "TEAM-ALPHA" } }</pre>

ImpactLevelUpdated

```
{
  "source": "MissionProgressService",
  "detail-type": "ImpactLevelUpdated",
  "detail": {
    "schema_version": "1.0",
    "mission_id": "MISS-a1b2c3d4",
    "incident_id": "INC-001",
    "rescue_team_id": "TEAM-ALPHA",
    "old_level": 2,
    "new_level": 4,
    "updated_at": "2025-06-14T09:35:00Z",
    "updated_by": "TEAM-ALPHA"
  }
}
```

⑤b Fallback: Outbox Pattern

EventBridge Publish ล้มเหลว



บันทึกลง EventOutbox Table (DynamoDB)

```
{
  "outbox_id": "OBX-uuid",
  "event_type": "MissionStatusChanged",
  "event_payload": "{...}",
  "status": "PENDING",
  "retry_count": 0,
  "ttl": "<7 days>"
}
```

สำคัญ: POST request ไม่ fail เพราะ EventBridge ล้มเหลว — ข้อมูลสถานะถูกบันทึกใน DynamoDB แล้ว

สรุป Interaction ทั้งหมด

Inbound (เข้า MissionProgress)

#	Source	ช่องทาง	Endpoint / Event
①	Rescue Team	Sync POST	POST /missions/{request_id}/progress
②	Rescue Team	Sync GET	GET /missions/{request_id}
③	ManageDispatch	Sync GET	GET /missions/{request_id}
⑥	Rescue Team	Sync POST	POST /missions/{request_id}/presigned-url
	Rescue Team	Sync POST	GET /missions/{request_id}/presigned-url?image_key=
⑦	Rescue Team	Sync GET	GET /missions (header: X-Rescue-Team-ID)
⑧	Dispatch Mgmt	Async (SNS)	DispatchOrderCreated (via SNS rescue.mission.dispatch.v1)

Downstream / Outbound (ออกจาก MissionProgress)

#	Destination	ช่องทาง	Event / API
④	RescueRequest	Sync GET	GET /v1/rescue-requests/{requestId}
④ _b	ManageDispatch	Sync GET	GET /v1/dispatches?teamId={teamId}
④ _c	RescueTeam	Sync GET	GET /v1/teams/{teamId}
④ _d	RescueTeam	Sync PATCH	PATCH /v1/teams/{teamId}/status (RESOLVED)
⑥ _b	Amazon S3	Sync	Generate Presigned URL
⑤ _a	IncidentTracking	Async Event	MissionStatusChanged + ImpactLevelUpdated
⑤ _b	ManageDispatch	Sync PATCH	PATCH /v1/dispatches/{id}/status (RESOLVED, fire-and-forget)
⑤ _c	Rescue Prioritization	Async Event	MissionBackupRequested + ImpactLevelUpdated

Frontend → S3 Direct (ไม่ผ่าน MissionProgress)

#	Source	Destination	ช่องทาง
⑥ _c	Rescue Team (Frontend)	Amazon S3	HTTP PUT (Presigned URL)

Downstream Services

#	Service	Owner	ช่องทาง	Interaction
1	RescueRequest Service	Phattharaphum Kingchai	HTTP GET	Sync GET (get-mission parallel + mission-assigned-handler)
2	ManageDispatch Service	Noppakron Songkroh	HTTP GET + SNS + Sync PATCH	Sync GET (get-mission) + Sync PATCH on RESOLVED + Inbound DispatchOrderCreated via SNS

3	RescueTeam Service	กมลพันธ์ กันธายอด	HTTP GET + PATCH	Sync GET (get-mission) + PATCH status on RESOLVED
4	IncidentTracking Service	Krittamet Damthongkam	EventBridge	Async (MissionStatusChanged + ImpactLevelUpdated)
5	Rescue Prioritization Service	Nattasak Chonmanat	EventBridge	Async (MissionBackupRequested + ImpactLevelUpdated)

Failure Handling

กรณี	การจัดการ
IncidentTracking ล้ม	Degraded Mode (data_source: "partial")
EventBridge Publish ล้มเหลว	Outbox Pattern → EventOutbox table
Lambda Authorizer ล้ม	HTTP 500
S3 Presigned URL ล้มเหลว	Lambda return 500 → User Retry
S3 Upload ล้มเหลว	อนุญาตให้ "ข้าม" → ส่งแค่ Text Status

Dependency Mapping

Dependency 1: IncidentTracking Service

Field	Value
Service Owner	Krittamet Damthongkam
Type	Service
Interaction Style	Hybrid (Synchronous + Asynchronous)
Criticality	High (รองรับ Degraded Mode)

Purpose

- ใช้ดึงข้อมูล Incident (description, location, type)
- เป็น Source of Truth ของ Incident

Interaction

ทิศทาง	ช่องทาง	รายละเอียด
ขาออก Async	EventBridge MissionStatusChanged	อัปเดตสถานะ Incident
ขาออก Async	EventBridge ImpactLevelUpdated	อัปเดต Impact Level

Failure Handling

กรณี	การจัดการ
Event Publish ล้มเหลว	Outbox Pattern — outbox-processor Lambda retry ทุก 5 นาที

Dependency 2: Dispatch Management Service

Field	Value
Service Owner	Noppakron Songkroh
Type	Service
Interaction Style	Sync + Async (Bidirectional)
Criticality	Critical

Interaction

ทิศทาง	ช่องทาง	รายละเอียด
ขาเข้า Async	SNS rescue.mission.dispatch.v1 DispatchOrderCreated	สร้าง Mission Record (ผ่าน mission-assigned-handler)
ขาออก Sync	PATCH /v1/dispatches/{id}/status	ปิด Dispatch Order เมื่อภารกิจ RESOLVED (fire-and-forget)
ขาออก Sync	GET /v1/dispatches?teamId=	ดึง dispatch status, priority_level (เสริม get-mission)
ขาเข้า Sync	GET /missions/{request_id}	Dispatcher ดู Timeline + รูปภาพ

Failure Handling

วิธี	คำอธิบาย
ไม่ได้รับ Event	ภารกิจไม่ถูกสร้าง (ตรวจสอบ SNS subscription + Lambda permission)
Publish ลมเหลว	Outbox Pattern — outbox-processor Lambda retry ทุก 1 นาที
PATCH RESOLVED ลม	Log WARN + ผ่าน (fire-and-forget, ไม่ block response)

Dependency 3: RescueTeam Service

Field	Value
Service Owner	กมลพันธ์ กันธายอด
Type	Service
Interaction Style	Synchronous (REST)
Criticality	High (รองรับ Degraded Mode)

Purpose

- ดึงข้อมูลทีม (team_name, team_type, capabilities, equipment, location)
- เป็น Source of Truth ของทีมกู้ภัย
- อัปเดตสถานะทีม เมื่อภารกิจ RESOLVED

Endpoints ที่เรียก

- GET /v1/teams/{teamId} — ดึงเอกสารทีม (Bearer auth)
- PATCH /v1/teams/{teamId}/status — อัปเดตสถานะทีม เป็น AVAILABLE (Bearer auth)

Interaction

ทิศทาง	ช่องทาง	Lambda	รายละเอียด
ขาออก Sync	GET /v1/teams/{teamId}	get-mission	ดึงเอกสารทีม (ล้มเหลว → Degraded Mode)
ขาออก Sync	PATCH /v1/teams/{teamId}/status	report-progress	เมื่อ RESOLVED → ปลดปล่อยทีม (fire-and-forget)

Failure Handling

กรณี	การจัดการ
GET ล้มเหลว (timeout 800ms + retry 2x)	Degraded Mode → omit team fields จาก response
PATCH ล้มเหลว	Non-blocking — fire-and-forget goroutine, ไม่ fail response

Dependency 4: Rescue Prioritization

Field	Value
Service Owner	Nattasak Chonmanat
Type	Service
Interaction Style	Async (Event-Driven)
Criticality	Medium (Non-blocking)

Interaction

ทิศทาง	Event	รายละเอียด
ขาออก Async	MissionBackupRequested	คำนวณ Priority ใหม่
ขาออก Async	ImpactLevelUpdated	อัปเดตลำดับความสำคัญ

Failure Handling

- Non-blocking → ระบบหลักยังทำงานได้
- EventBridge มี retry 24 ชม.
- ใช้ Outbox Pattern เป็น safety net

Dependency 5: Amazon S3 (Evidence Storage)

Field	Value
Type	Infrastructure (Object Storage)
Interaction Style	Synchronous
Criticality	Medium

Purpose

- เก็บรูปภาพหลักฐาน
- Upload ผ่าน Presigned URL

Failure Handling

กรณี	การจัดการ
Generate URL ล้มเหลว	Retry
Upload ล้มเหลว	ข้ามได้ → ส่ง Text ก่อน
S3 ล้ม	Degrade → ใช้ Text only

Dependency 6: RescueRequest Service

Field	Value
Service Owner	Phattharaphum Kingchai
Type	Service
Interaction Style	Synchronous (REST)
Criticality	High (รองรับ Degraded Mode)

Purpose

- ดึงข้อมูล Request (description, location, requestType, peopleCount)
- เป็น Source of Truth ของ Rescue Request ที่ผูกกับภารกิจนี้

Endpoint ที่เรียก

GET /v1/rescue-requests/{requestId}

- Auth: Authorization: Bearer <token>

- รับ requestId จาก path parameter ของ GET /missions/{request_id}

Interaction

ทิศทาง	ช่องทาง	รายละเอียด
ขาออก Sync	HTTP GET /v1/rescue-requests/...	ดึงข้อมูล Request (ล้มเหลว → Degraded Mode)

เรียกในทั้ง get-mission Lambda (parallel) และ mission-assigned-handler Lambda

Failure Handling

กรณี	การจัดการ
Sync GET ล้มเหลว	Degraded Mode → ส่งเฉพาะข้อมูลที่มี (data_source: partial)

หมายเหตุ: Service URL และ Bearer token configure ผ่าน Lambda environment variable (RESCUE_REQUEST_SERVICE_URL, SERVICE_BEARER_TOKEN) — ไม่ hardcode ใน code

Dependency 7: API Gateway + Authorizer

Field	Value
Type	Infrastructure (Auth)
Interaction Style	Sync
Criticality	Critical

Purpose

- รับ HTTP Request ทั้งหมด
- ตรวจสอบ x-api-key, X-Rescue-Team-ID

Failure Handling

กรณี	การจัดการ
API Key ผิด	403
ไม่มี Header	403
Authorizer ล่ม	500
API Gateway ล่ม	ใช้ local cache

Dependency 7: Amazon EventBridge

Field	Value
Type	Event Bus
Bus Name	mission-progress-events
Interaction	Async
Criticality	Critical

Events

Event	Trigger	สถานะปัจจุบัน
MissionStatusChanged	สถานะเปลี่ยน	CloudWatch Logs (IncidentTracking: EventBridge→Lambda direct; Dispatch: sync PATCH)
MissionBackupRequested	NEED_BACKUP	CloudWatch Logs
ImpactLevelUpdated	มี new_impact_level	CloudWatch Logs

Failure Handling

ระดับ 1: Publish ล้มเหลว

- ใช้ Outbox Pattern — outbox-processor Lambda retry ทุก 5 นาที

ระดับ 2: Delivery ล้มเหลว

- EventBridge retry อัตโนมัติ (24 ชม.)
- รองรับ DLQ

Dependency 8: RescueTeam Service

Field	Value
Service Owner	กมลพันธุ์ กันธายอด
Type	Service
Interaction Style	Synchronous (REST)
Criticality	Medium (รองรับ Degraded Mode)

Purpose

- ดึงข้อมูลทีมกู้ภัย (ชื่อ, ความสามารถ) เพื่อเสริมข้อมูลใน get-mission response

Interaction

ทิศทาง	ช่องทาง	รายละเอียด
ขาออก Sync	HTTP GET /teams	ดึงข้อมูลทีม (ล้มเหลว → Degraded Mode)

Failure Handling

กรณี	การจัดการ
Sync GET ล้มเหลว	Degraded Mode → ส่งเฉพาะข้อมูลที่มี (data_source: partial)